Assignment-5

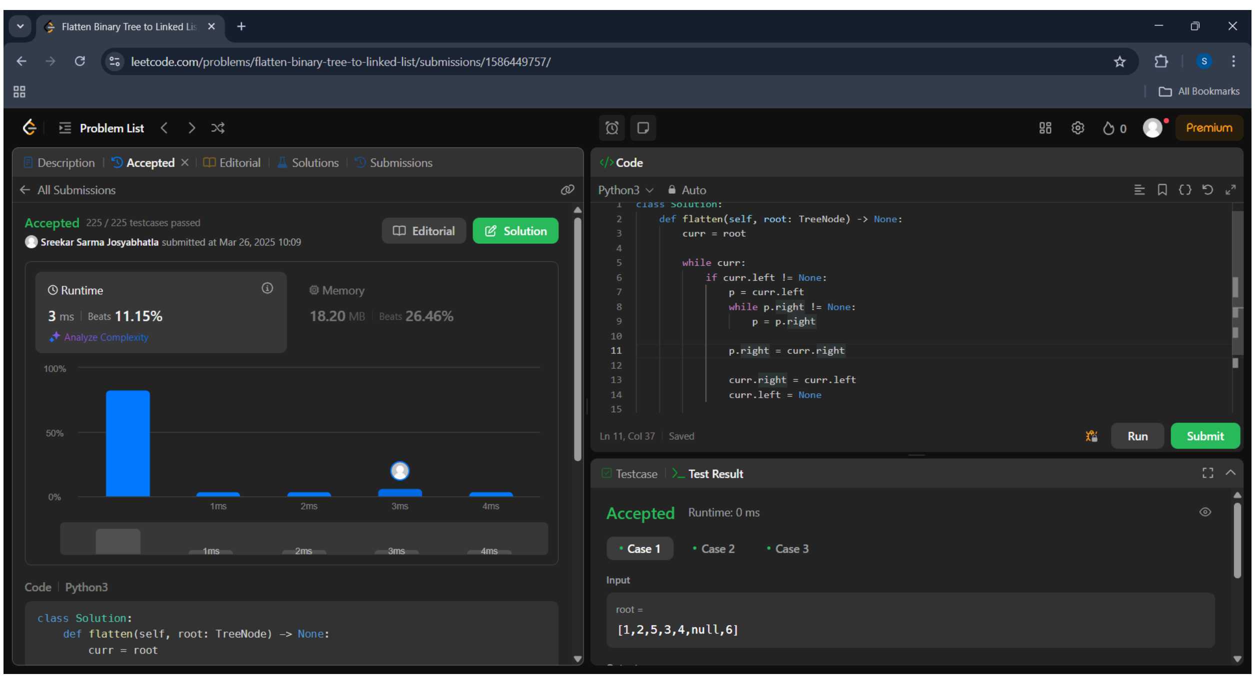
J.Sreekar Sarma
VU21CSEN0300040

1. Flatten Binary Tree to Linked List

Python

```python
class Solution:
    def flatten(self, root: TreeNode) -> None:
        curr = root

        while curr:
            if curr.left != None:
                p = curr.left
                while p.right != None:
                    p = p.right

                p.right = curr.right

                curr.right = curr.left
                curr.left = None

            curr = curr.right
```

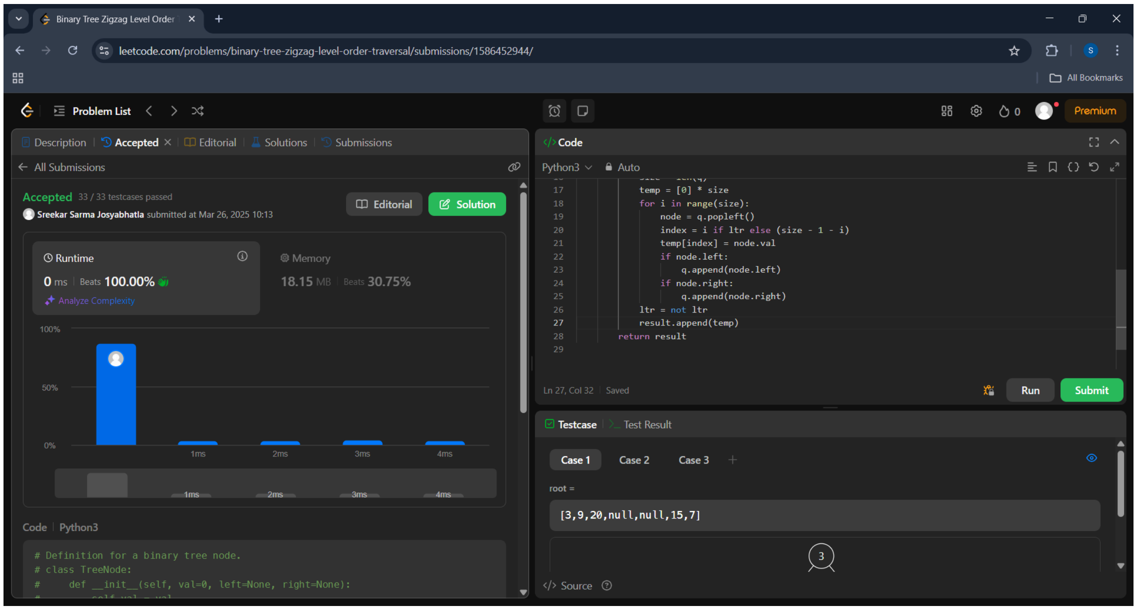
OUTPUT



2. Binary Tree Zigzag Level Order Traversal

Python
```python
class Solution:
    def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:

        if not root:
            return []
        result = []
        q = deque([root])
        ltr = True
        while q:
            size = len(q)
            temp = [0] * size
            for i in range(size):
                node = q.popleft()
                index = i if ltr else (size - 1 - i)
                temp[index] = node.val
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ltr = not ltr
            result.append(temp)
        return result
```

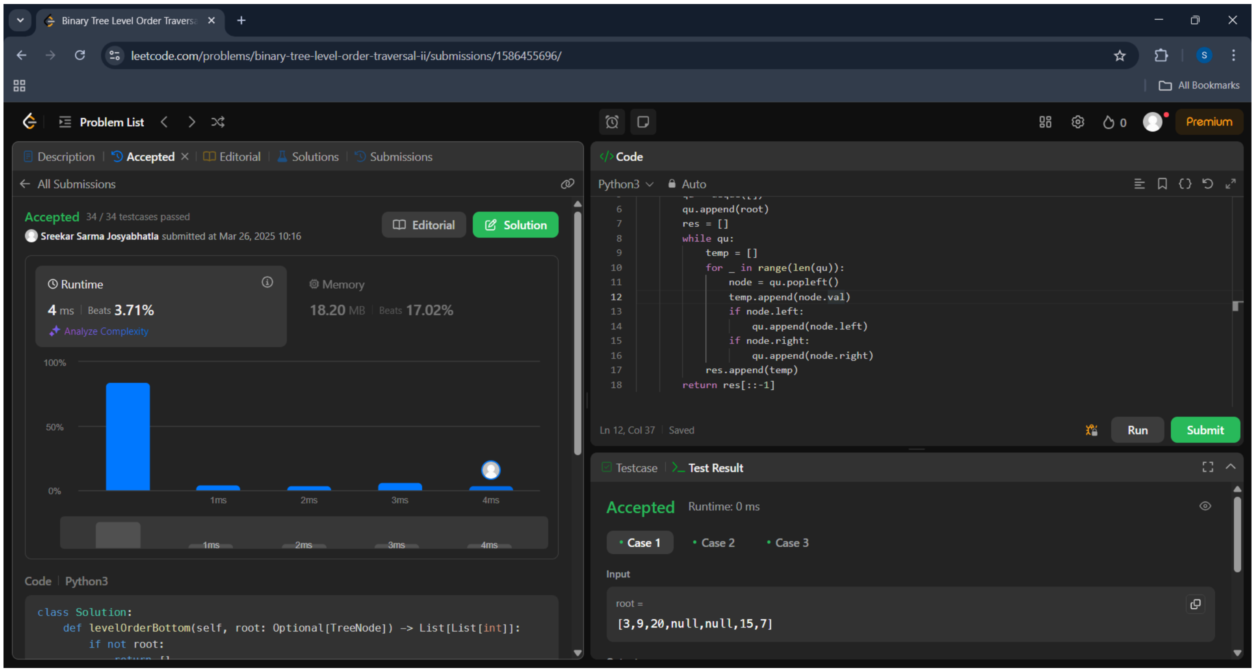
OUTPUT



3. Binary Tree Level Order Traversal II

Python
```python
class Solution:
    def levelOrderBottom(self, root: Optional[TreeNode]) -> List[List[int]]:
        if not root:
            return []
        qu = deque([])
        qu.append(root)
        res = []
        while qu:
            temp = []
            for _ in range(len(qu)):
                node = qu.popleft()
                temp.append(node.val)
                if node.left:
                    qu.append(node.left)
                if node.right:
                    qu.append(node.right)
            res.append(temp)
        return res[::-1]
```

OUTPUT

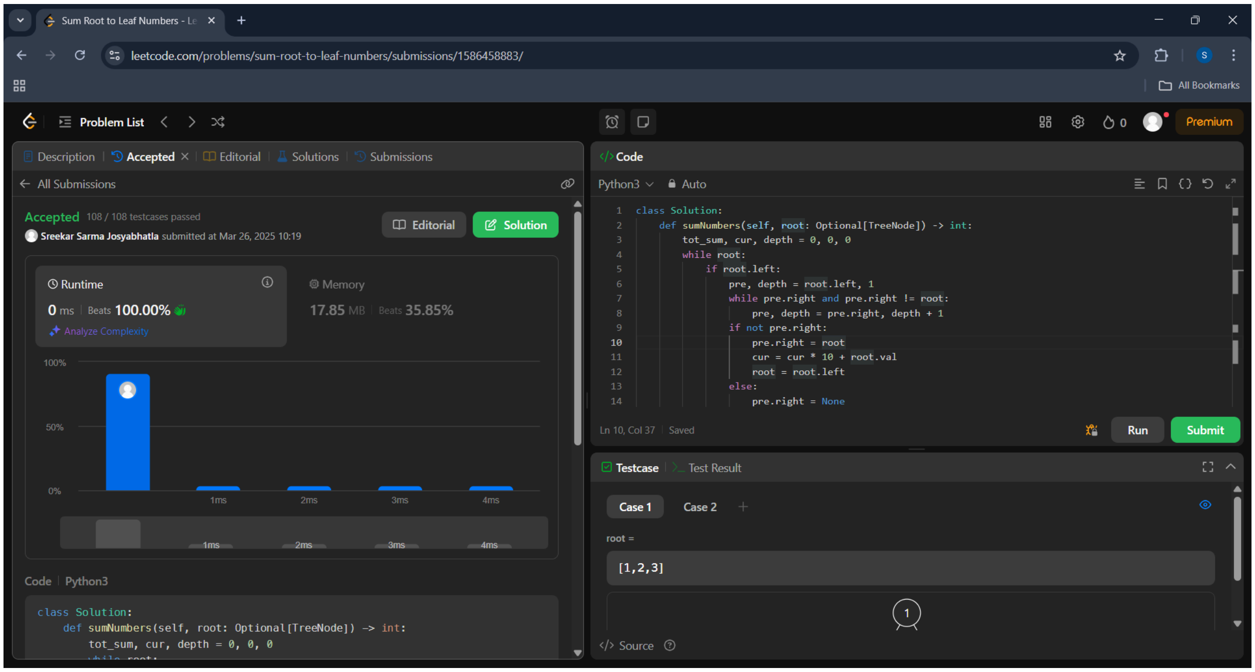


4. Sum Root to Leaf Numbers

Python
```python
class Solution:
    def sumNumbers(self, root: Optional[TreeNode]) -> int:
        tot_sum, cur, depth = 0, 0, 0
        while root:
            if root.left:
                pre, depth = root.left, 1
                while pre.right and pre.right != root:
                    pre, depth = pre.right, depth + 1
                if not pre.right:
                    pre.right = root
                    cur = cur * 10 + root.val
                    root = root.left
                else:
                    pre.right = None
                    if not pre.left: tot_sum += cur
                    cur //= 10**depth
                    root = root.right
            else:
                cur = cur * 10 + root.val
                if not root.right: tot_sum += cur
```

```
            root = root.right
    return tot_sum
```

OUTPUT



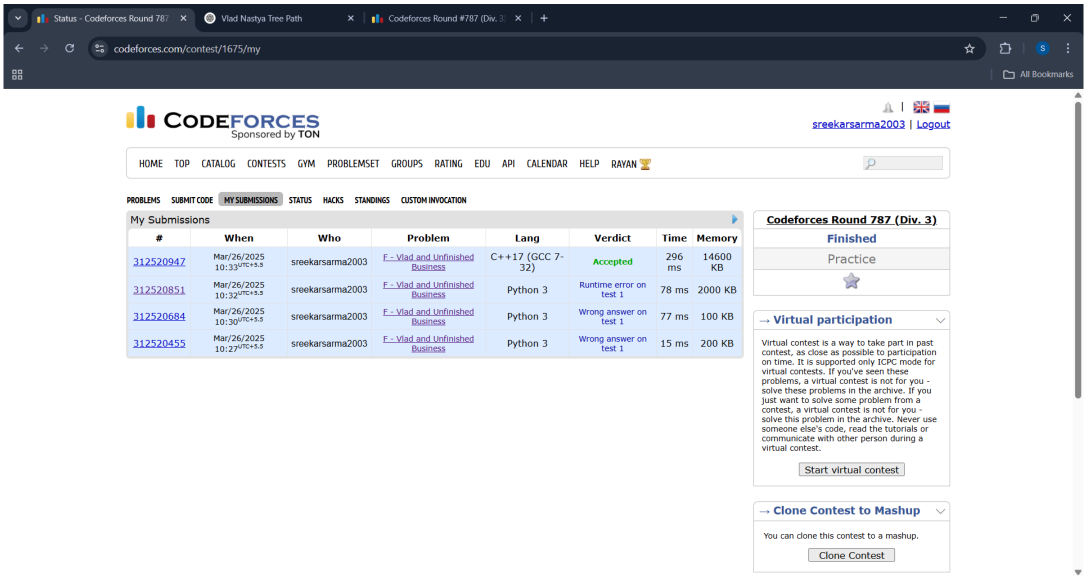
5. Vlad and Unfinished Business

C/C++
```
#include <bits/stdc++.h>

using namespace std;

//#define int long long
#define ll long long
//#define double long double
#define all(a) a.begin(), a.end()
#define rall(a) a.rbegin(), a.rend()

const int MOD = 1e9 + 7;
const int maxN = 5e3 + 1;
const int INF = 2e9;
```

```cpp
const int MB = 20;

vector<vector<int>> g;
vector<bool> todo, good;

void dfs(int v, int p = -1) {
    for (int u : g[v]) {
        if (u != p) {
            dfs(u, v);
            if (todo[u]) {
                todo[v] = true;
            }
            if (good[u]) {
                good[v] = true;
            }
        }
    }
}

void solve() {
    int n, k;
    cin >> n >> k;
    g.clear();
    g.resize(n);
    int x, y;
    cin >> x >> y;
    --x;
    --y;
    todo.resize(n);
    fill(all(todo), false);
    good.resize(n);
    fill(all(good), false);
    for (int i = 0; i < k; ++i) {
        int v;
        cin >> v;
        --v;
        todo[v] = true;
    }
    good[y] = true;
    for (int i = 0; i < n - 1; ++i) {
```

```
        int v, u;
        cin >> v >> u;
        --v;
        --u;
        g[v].push_back(u);
        g[u].push_back(v);
    }
    dfs(x);
    int ans = 0;
    for (int i = 0; i < n; ++i) {
        if (i == x) {
            continue;
        }
        if (good[i]) {
            ++ans;
        } else if (todo[i]) {
            ans += 2;
        }
    }
    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    //    srand(time(0));
    int t = 1;
    cin >> t;
    while (t--) solve();
}
```

OUTPUT:



6. G. Remove Directed Edges

C/C++
```cpp
#include <bits/stdc++.h>

#define forn(i, n) for (int i = 0; i < int(n); i++)

using namespace std;

const int INF = 1e9;

vector<int> in, out;
vector<vector<int>> g;
vector<int> dp;

int calc(int v){
    if (dp[v] != -1) return dp[v];
    if (in[v] >= 2 && out[v] >= 2){
        dp[v] = 1;
        for (int u : g[v])
            dp[v] = max(dp[v], calc(u) + 1);
    }
    else if (in[v] >= 2){
        dp[v] = 1;
    }
    else{
```

```
        dp[v] = -INF;
    }
    return dp[v];
}

int main() {
    int n, m;
    scanf("%d%d", &n, &m);
    g.resize(n);
    in.resize(n);
    out.resize(n);
    forn(i, m){
        int v, u;
        scanf("%d%d", &v, &u);
        --v, --u;
        g[v].push_back(u);
        ++in[u];
        ++out[v];
    }
    int ans = 1;
    dp.resize(n, -1);
    forn(v, n) if (out[v] >= 2){
        for (int u : g[v]){
            ans = max(ans, calc(u) + 1);
        }
    }
    printf("%d\n", ans);
    return 0;
}
```

OUTPUT

312521384	Mar/26/2025 10:38 UTC+5.5	sreekarsarma2003	1674G - Remove Directed Edges	C++17 (GCC 7-32)	Accepted	218 ms	3200 KB

7. Fall Down

```
C/C++
    #include <iostream>
    #include <vector>
```

```
int main(){

    std::ios_base::sync_with_stdio(false);
    long t; std::cin >> t;
    while(t--){
        long nr, nc; std::cin >> nr >> nc;
        std::vector<std::string> v(nr);
        for(long row = 0; row < nr; row++){std::cin >> v[row];}

        for(long col = 0; col < nc; col++){
            long b(nr - 1);
            for(long row = nr - 1; row >= 0; row--){
                if(v[row][col] == '*'){
                    v[row][col] = '.';
                    v[b][col]   = '*';
                    --b;
                }
                else if(v[row][col] == 'o'){b = row - 1;}
            }
        }

            for(long row = 0; row < nr; row++){std::cout << v[row] << std::endl;}
    }

}
```

Output:

312521569	Mar/26/2025 10:41UTC+5.5	sreekarsarma2003	1669G - Fall Down	C++17 (GCC 7-32)	Accepted	62 ms	0 KB

8. 1670C - Where is the Pizza?

C/C++

```
#include <bits/stdc++.h>

using namespace std;
const int N = 100100;
int n;
```

```
int a[N], b[N], c[N];
bool used[N];
int par[N], sz[N];

int getPar(int v) {
	return par[v] == v ? v : (par[v] = getPar(par[v]));
}
bool unite(int v, int u) {
	v = getPar(v);
	u = getPar(u);
	if (v == u) return false;
	if (sz[v] < sz[u]) swap(v, u);
	sz[v] += sz[u];
	par[u] = v;
	return true;
}

void solve() {
	scanf("%d", &n);
	for (int i = 1; i <= n; i++) {
		used[i] = 0;
		par[i] = i;
		sz[i] = 1;
	}
	for (int i = 0; i < n; i++)
		scanf("%d", &a[i]);
	for (int i = 0; i < n; i++)
		scanf("%d", &b[i]);
	for (int i = 0; i < n; i++) {
		scanf("%d", &c[i]);
		if (c[i] != 0) used[c[i]] = 1;
	}
	long long ans = 1,mod=1e9+7;
	for (int i = 0; i < n; i++) {
		if (c[i] != 0) continue;
		int v = a[i], u = b[i];
		if (used[v] || used[u] || v == u) continue;
		if (!unite(v, u)) ans = (ans*2)%mod;
	}
	printf("%u\n", ans);
}


int main()
```

```
{

    int t;
    scanf("%d", &t);
    while(t--) solve();

    return 0;
}
```

Output:

312522057	Mar/26/2025 10:46 UTC+5.5	sreekarsarma2003	1670C - Where is the Pizza?	C++17 (GCC 7-32)	Accepted	124 ms	2100 KB